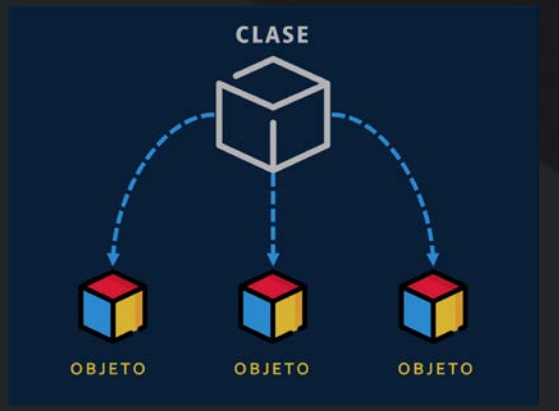




Programacion Orientada a objetos

2

CLASES

Conceptualmente, las clases son el componente principal del paradigma de la (Programación Orientada A Objetos). En donde representan entidades o conceptos, que forman parte de un modelo predefinido.

En Kotlin, una clase es una agrupación de propiedades (atributos) y funciones (métodos) miembro, que actúan como una plantilla para la instanciación de objetos.

La declaración de una clase se compone de:

- Nombre de la clase
- Cabecera
- Cuerpo

Para definirla, se usa la palabra clave **class** seguida de su nombre. Opcionalmente, puede llevar más elementos como:

- Modificadores. Controlan la visibilidad y el comportamiento (open, abstract, private, etc.).
- Declaración de parámetros
- Constructor primario. Define los parámetros utilizados para inicializar la clase y declarar las propiedades (atributos).
- Herencia. Indica de qué otra clase o interfaz hereda (: NombreDeLaClasePadre).
- Restricciones de tipo

CLASES: class y data class

En Kotlin podemos tener **clases estándar** (class) o **clases de datos** (data class). Aunque ambas sirven como plantillas para crear instancias, se diferencian en su propósito y en las funcionalidades que el compilador genera automáticamente.

La elección entre una u otra no es arbitraria: el tipo que escojamos define cómo se comportarán los objetos al ser comparados, mostrados o copiados.

- Una **class** es una plantilla genérica para para modelar entidades con lógica compleja y comportamiento. Con ellas creamos objetos con comportamiento y estado.
- Una **data class** es una clase especializada, optimizada para ser un simple **contenedor de datos.**

Las data class en Kotlin **no pueden ser clases padre para herencia**, es decir, no pueden tener clases hijas. Sin embargo, **si pueden implementar interfaces**.

CLASES: **data class**

Se utilizan para clases cuya función principal es almacenar información y ser utilizadas en comparaciones o visualizaciones de manera legible. Kotlin genera automáticamente varios métodos esenciales basados en las propiedades declaradas en el constructor primario:

- **equals().** Compara dos objetos basándose en los valores de sus propiedades (igualdad estructural). Es decir, puedo comparar dos objetos usando '==' y Kotlin intenamente lo traduce a una llamada a equals.
- **hashCode().** Genera un código hash basado en los valores de las propiedades. Necesario para colecciones como HashMap y HashSet.
- **toString().** Devuelve una representación legible de la clase, incluyendo los nombres y valores de todas las propiedades (ej.: Usuario(id=1, nombre=Ana)).
- **componentN().** Permite la desestructuración de objetos (asignar propiedades a variables separadas de forma concisa).
- **copy().** Permite crear una copia del objeto, modificando opcionalmente algunas de sus propiedades.

En el desarrollo de Android, todos los objetos que representan entidades (usuarios, productos, ítems de una lista) deben ser definidos como data class.

CLASES: **data class**

Ejemplo

```
data class Usuario(val id: Int, val nombre: String)
```

```
fun main() {
    val usuarioOriginal = Usuario(id = 1, nombre = "Ana")

    // Uso de toString() : Se usa automáticamente para una salida legible.
    println("Usuario Original: $usuarioOriginal")

    //Uso de copy() : Crea una copia exacta, modificando el ID.
    val usuarioDuplicado = usuarioOriginal.copy()
    println("Usuario Duplicado: $usuarioDuplicado")
    val usuarioModificado = usuarioOriginal.copy(id = 2)
    println("Usuario Modificado: $usuarioModificado")

    //Uso de equals() : Comprueba el contenido de las propiedades, no la referencia.
    println("¿Son el original y el duplicado iguales? ${ if (usuarioOriginal == usuarioDuplicado) "si" else "no"}")
    println("¿El original y el modificado son iguales? ${if (usuarioOriginal == usuarioModificado) "si" else "no"}")

    // Uso de componentN() : permite asignar las propiedades a variables locales en el orden en que fueron declaradas.
    val (identificador, nombreUsuario) = usuarioOriginal
    println("ID: $identificador")
    println("Nombre: $nombreUsuario")
}
```

CLASES: **class**

Las clases estándar (class) en Kotlin tienen por defecto los mismos métodos que la clase base java.lang.Object (ya que todas las clases Kotlin heredan de Any, que es su equivalente para la JVM). Por lo tanto, una clase estándar sin ninguna implementación adicional (sin ser data class ni sobrescribir nada) tiene por defecto los siguientes tres métodos:

- **equals().** Realiza una comparación referencial (lo mismo que el operador ===). Solo devuelve true si ambas variables apuntan exactamente a la misma instancia en la memoria. No compara el contenido de las propiedades.
- **hashCode().** Devuelve un valor único basado en la dirección de memoria del objeto. Esto es necesario para el uso de colecciones como HashMap y HashSet.
- **toString().** Muestra el nombre de la clase, seguido del símbolo @ y el código hash de la instancia (por ejemplo: MiClase@1a2b3c4d). No muestra una representación legible.

Una class estándar obliga al desarrollador a sobrescribir (override) estos tres métodos si se desea que la clase se comporte como un contenedor de datos (es decir, si se quiere que == compare el contenido). En cambio, una data class genera automáticamente implementaciones útiles para toString(), equals() y hashCode() que basan su lógica en las propiedades declaradas en el constructor primario.

CONSTRUCTORES

Un constructor es una función especial que se usa para inicializar el contenido de las nuevas instancias de una clase.

A diferencia de otros lenguajes, **no se usa la palabra new** para crear el objeto, sino que se llama al constructor como una función normal.

Características:

- Es el primer método que se ejecuta.
- Se ejecuta en forma automática.
- No puede retornar datos.
- Se ejecuta una única vez.
- Un constructor tiene por objetivo inicializar atributos.
- Una clase puede tener varios constructores, pero solo uno es el primario. Si no se crea un constructor, entonces el compilador de Kotlin genera uno sin parámetros por defecto.

CONSTRUCTORES

Constructor Primario

El constructor primario no tiene cuerpo propio y su objetivo es declarar las propiedades y los parámetros que se necesitan para inicializar un objeto.

Las asignaciones de los parámetros a las propiedades se realizan automáticamente por el compilador.

Para que un parámetro se convierta en una propiedad accesible de la clase, se deben usar las palabras clave **val** o var (mutable) delante de él.

```
class Coche(val marca: String, val anio: Int) {
    var encendido: Boolean = false
    fun arrancar() {…}

}
```

CONSTRUCTORES

Constructores Secundarios

Los constructores secundarios se utilizan para proporcionar formas alternativas de inicializar la clase. Se declaran dentro del cuerpo de la clase utilizando la palabra clave constructor.

Si una clase tiene un constructor primario, todo constructor secundario **debe llamar al constructor primario** de forma directa o indirecta mediante la expresión **this()**. Con esta regla aseguramos que todas las propiedades definidas en el constructor primario se inicialicen correctamente

```
class Coche(val marca: String, val anio: Int) {

    constructor(marca: String) : this(marca, 2024) {
    println("Inicializo la clase con año por defecto.")
    }
}
```

CONSTRUCTORES

Modificadores de visibilidad

Los modificadores de un constructor en Kotlin controlan su visibilidad y, por lo tanto, quién puede crear instancias de esa clase. Son los mismos modificadores que se utilizan en la programación orientada a objetos (POO) para controlar la visibilidad de otros miembros de la clase (propiedades y funciones/métodos).

Se utilizan para limitar el acceso al constructor o al miembro (método o propiedad) desde otras partes del código.

Son los mismos modificadores que se usan con los métodos (fun), la única diferencia es que ***protected*** no está permitido para el constructor primario. Si yo declarase el constructor primario protected, significaría que solo las subclases pueden acceder a él. Como el constructor primario se usa a menudo para crear objetos, está restringido. Sin embargo, sí se puede usar protected en un constructor secundario si en el código de este constructor secundario se delega o hace uso correctamente del constructor primario.

CONSTRUCTORES

Modificadores de visibilidad

Modificador	Acceso en el Constructor	Acceso en el Método/Propiedad	Ejemplo
public (Por defecto)	Accesible desde cualquier lugar. Cualquier código puede instanciar la clase.	Accesible desde cualquier lugar.	class PublicClass(val id: Int) fun obtenerDatos() { ... }
internal	Accesible solo dentro del mismo módulo (o componente de software).	Accesible solo dentro del mismo módulo (o componente de software).	internal class InternalRepo() internal fun iniciarServicio() { ... }
protected	No se permite para el constructor primario. Solo en secundarios.	Accesible dentro de la clase y por las subclases (clases hijas).	protected fun mostrarDetalles() { ... }

private	Accesible solo dentro de la propia clase. Evita que se creen instancias desde fuera (patrón Singleton).	Accesible solo dentro de la propia clase.	class Clase private constructor(private val claveSecreta: String) {
			fun getClave(): String return claveSecreta
			}

CONSTRUCTORES

El bloque init

Se ejecuta **como parte del constructor primario**. Su código se ejecuta inmediatamente después de que las propiedades de la cabecera han sido inicializadas, pero antes de que se ejecute la lógica de cualquier constructor secundario. Se usa principalmente para:

- Validación de propiedades (por ejemplo, asegurar que un número es positivo).
- Lógica compleja que necesita ejecutarse en el momento de la creación de un objeto.

Aunque el bloque init está asociado al constructor primario, se pueden declarar varios bloques init dentro del cuerpo de una clase. El compilador de Kotlin simplemente los trata como un único bloque de inicialización y los ejecuta secuencialmente en el orden en que aparecen en el código.

```
class Coche(val marca: String, val anio: Int) {
      init {
     if (anio <= 0) {
               throw IllegalArgumentException("El año de fabricación debe ser un valor positivo.")
         }
      }
}
```

CONSTRUCTORES

El método require()

Es una función estándar de Kotlin que se utiliza para validar argumentos o estados de una función o un constructor. Se usa para asegurar que una condición sea verdadera.

Si la condición dentro de require() es falsa, el método lanza una excepción de tipo **IllegalArgumentException**, y detiene la ejecución del código.

Se usa de forma muy habitual dentro del bloque **init** en un **constructor primario** para validar los datos que se pasan durante la creación de un objeto. Esto ayuda a implementar las reglas de negocio de forma que el objeto solo se crea si cumple estas condiciones.

Se puede considerar que el require() es solo un atajo de Kotlin para lanzar una *IllegalArgumentException* cuando una condición que se le pasa como parámetro no se cumple.

CONSTRUCTORES

Si reescribimos el código del **init** anterior con **require()** quedaría así:

```
class Coche(val marca: String, val anio: Int) {
      init {
     if (anio <= 0) {
               throw IllegalArgumentException("El año de fabricación debe ser un valor positivo.")
         }
      }
}
```

```
class Coche(val marca: String, val anio: Int) {
      init {
     require(anio > 0) { "El año de fabricación debe ser un valor positivo." }
         }
      }
}
```

CLASES

```
class Coche1(val marca: String, val anio: Int) {
      var encendido: Boolean = false

      constructor(marca: String) : this(marca, 2024) {
            // constructor secundario
            println("Creando coche de marca $marca del año $anio.")
      }

      fun arrancar() {
            if (!encendido) {
                  encendido = true
                  println("$marca del $anio ha arrancado.")
            } else {
                  println("El coche ya está encendido.")
            }
      }
}
```

Clase con constructor primario y secundario

```
fun main() {
      val coche1 = Coche1("Toyota", 2020)
      val coche2 = Coche1("Toyota")
      coche1.arrancar()
      coche2.arrancar()
}
// Creando coche de marca Toyota del año 2020.
// Creando coche de marca Toyota del año 2024.
// Toyota del 2020 ha arranado.
// Toyota del 2024 ha arranado.
```

```
class Coche2(val marca: String, val anio: Int? = 2024)
{
    var encendido: Boolean = false
    fun arrancar() {//mismo código}

}
```

Clase con constructor primario y valor por defecto

```
fun main() {
      val coche1 = Coche2("Toyota", 2020)
      val coche2 = Coche2("Toyota")
      Val coche3 = Coche2("Toyota“, null)
      coche1.arrancar()
      coche2.arrancar()
      Coche3.arrancar()
}
// Toyota del 2020 ha arranado.
// Toyota del 2024 ha arranado.
// Toyota del 2024 ha arranado.
```

Propiedades de clases: getters y setters

En Kotlin, las propiedades (properties) son variables de la clase que referencian tanto el atributo(donde se almacena el dato) como sus métodos de acceso (los getters y setters).

En Kotlin, los getters y setters se generan y llaman automáticamente para las propiedades val (se genera sólo getter) y var (se generan getter y setter), no necesitamos escribirlos. Usaremos el operador punto y el operador de asignación.

```
class Producto(val nombre: String, var precio: Double) {…}
```

Acción	Sintaxis en Kotlin	Sintaxis equivalente en Java
Lectura (Getter)	`producto.precio`	`producto.getPrecio()`
Escritura (Setter)	`producto.precio = 50.0`	`producto.setPrecio(50.0)`

Propiedades de clases: getters y setters

En los getters y los setters no podemos usar el operador **this** para asignar un valor al un atributo sobre el que se hace el get o el set. Sólo podemos usarlo para leer valores de otros atributos. Si intentamos usarlo para asignarle un valor, se produce una recursión infinita (StackOverflowError).

Para realizar esa asignación Kotlin tiene el identificador **field** (llamado "campo de respaldo" o backing field) es una palabra reservada de Kotlin que solo está disponible dentro de los getters y setters personalizados.

field almacena el valor real de la propiedad en memoria.

Al usar field = valor, se le está asignando el valor directamente al espacio de memoria, sin invocar el setter nuevamente.

Operador	Uso	Razón
field (Campo de Respaldo)	Se usa para leer o escribir el valor de la propiedad **dentro de su propio get() o set().**	Accede directamente al espacio de memoria (field) donde está guardado el dato, sin volver a llamar al getter o setter. Esto evita la recursión infinita.
this.propiedad	Se usa para leer el valor de otras propiedades de la clase.	Llama de forma indirecta al getter de esa otra propiedad.

Propiedades de clases: getters y setters

Getters y setters por defecto

En esencia, declarar una propiedad genera sus getter con el identificador ***field*** internamente.

```
class Animal{
    var peso: Double = 0.0
}
```

Es equivalente

```
class Animal{
    var peso: Double = 0.0
       get() = field
       set(value) {
          field = value
       }
}
```

Propiedades de clases: getters y setters

Se puede modificar la visibilidad de los getters y setters anteponiéndoles un modificador de Visibilidad (por defecto son public).

```
class Animal{
   var peso: Double = 0.0
       get() = field
       private set(value) {
          field = value
       }
}
```

Lo que ocurre en el ejemplo es que entonces no se podría modificar nunca el peso del animal desde fuera de la clase.

En este caso, si necesitamos controlar la actualización del atributo, haremos el set por defecto privado y generaremos métodos para modificarlo (encapsulación).

```
class Animal {
    var peso: Double = 0.0
        private set

    fun crecer(kilos: Double) {
        //para decrecer poner kilos < 0
        peso += kilos
    }

}
```

Propiedades de clases: getters y setters

Getters y setters personalizados (custom)

Si necesitamos añadir lógica adicional a estos métodos, se pueden sobreescribir en el momento de la definición de los atributos.

Quitaremos de la cabecera **var/val** para definir los atributos dentro de la clase.

Cada vez que hagamos uso de la expresión, objeto.propiedad, automáticamente se ejecutará el código personalizado del getter.

Cada vez que asignemos un valor a la propiedad de un objeto, se ejecutará el setter que hayamos personalizado.

Ojo, si el atributo es de tipo var, podermos definir setter y getter, pero si es de tipo val, sólo podremos customizar el getter.

Propiedades de clases: getters y setters

```
class Producto(nombreInicial: String, precioBase: Double) {

    var nombre: String = nombreInicial
        get() = field.uppercase() //Devuelve el nombre en mayúsculas.
        set(valor) {
            if (valor.isBlank()) { //Evita nombres vacíos o nulos
                println("El nombre del producto no puede estar vacío.")
            } else {
                field = valor //field se refiere a nombre. No podemos usar this.nombre = valor
            }
        }

    var precio: Double = precioBase
        get() = field * 1.21 // Se ejecuta cada vez que se lee 'producto.precio'. Devuelve el precio base + 21%(IVA)

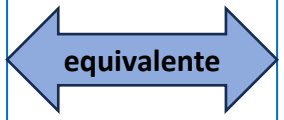
        set(valor) { //Se ejecuta cada vez que se asigna un nuevo valor a 'producto.precio'
            if (valor >= 0) {//validamos que el valor asignado sea siempre positivo.
                field = valor //field se refiere a precio, que es el campo que estamos definiendo
            } else {
                println("El precio no puede ser negativo. Se mantiene el precio anterior.")
            }
        }
}
```

Propiedades de clases: getters y setters

Ejemplo: este **getter** personalizado en el atributo esAdulto sirve para determinar si un animal es adulto, es decir, tiene tiene más de 3 años de vida.

```
class Animal(val nombre: String, val edad: Int) {

    val esAdulto: Boolean
     get() {
        return this.edad > 3
     }
}
```




```
class Animal(val nombre: String, val edad: Int) {

    val esAdulto: Boolean
     get() this.edad > 3
}
```

```
fun main(){
    val cachorro = Animal("Firulais", 2)
    val perroAdulto = Animal("Rex", 5)

    println("${cachorro.nombre} tiene ${cachorro.edad} años. ¿Es adulto? ${if (cachorro.esAdulto) "SI" else "NO"}")
    println("${perroAdulto.nombre} tiene ${perroAdulto.edad} años. ¿Es adulto? ${if (perroAdulto.esAdulto) "SI" else "NO"}")

}
//Salidas:
//Firulais tiene 2 años. ¿Es adulto? NO
//Rex tiene 5 años. ¿Es adulto? SI
```

Propiedades de clases: getters y setters

Ejemplo: este **setter** personalizado en el atributo peso sirve para asignar un valor al atributo y simultáneamente determinar si un animal tiene sobrepeso, es decir, pesa más de 50 kilos, actualizando el atributo sobrepeso.

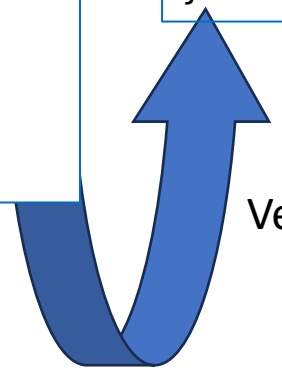
```
class Animal(val nombre: String, val edad: Int) {

    val esAdulto: Boolean
     get() {
        return this.edad > 3
     }

    var peso: Double = 0.0
     set(valor) {
         field = valor
         sobrepeso = valor > 50.0}

    var sobrepeso: Boolean = false

}
```



Versión mejorada

```
class Animal(val nombre: String, val edad: Int) {

    val esAdulto: Boolean
        get() = this.edad > 3

    var peso: Double = 0.0

    val sobrepeso: Boolean
        get() = this.peso > 50.0

}
```

Herencia

La herencia es un mecanismo de la Programación Orientada a Objetos, para diseñar dos o más entidades que son diferentes, pero comparten muchas características en común.

El proceso consiste en definir una clase conocida como: clase base, clase padre o superclase que contenga todas las características en común.

Luego se definen las clases que heredan las características, denominadas subclases, clases hijas, clases derivadas o clases descendientes.

Como ya hemos visto, la clase **Any** es la raíz de la jerarquía de clases en Kotlin. Cada clase en el lenguaje derivará de ella si no se especifica una superclase.

Herencia

Para ver el contenido de la clase Any, haremos una declaración en cualquier main del estilo:

```
val a: Any? = null
```

Luego nos situaremos sobre el nombre de la clase y pulsaremos el botón derecho. En las opciones buscamos Go To -> Declaration or Usages. Se abrirá el fichero Any.kt y podremos ver la declaración de los tres métodos básicos de la POO que a menudo sobrescribiremos (override) en nuestras propias clases de datos:

- **equals**(otroObjeto: Any?): Boolean. Determina si dos objetos tienen el mismo valor (igualdad estructural).
- **hashCode**(): Int. Proporciona un código único para el objeto.
- **toString**(): String. Devuelve una representación *legible* del objeto (Nombre_Completo_de_la_Clase@Código_Hash_del_Objeto_en_Hexadecimal)

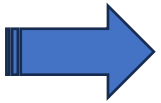
Herencia

Ejemplo:

```
fun main(){

    val animal1 = Animal("animal1", 1)
    val animal2 = Animal("animal2", 2)

    println(animal1.toString())
    println(animal2.toString())
    println(animal1.hashCode())
    println(animal2.hashCode())
    println(animal1.equals(animal1))
    println(animal1.equals(animal2))
}
```



```
Salida si no he sobreescrito nada en Animal

pruebas1.Animal@63961c42
pruebas1.Animal@65b54208
1670782018
1706377736
true
false
```

Herencia

En Kotlin, todas las clases y sus miembros (funciones y propiedades) son ***final*** por defecto. Esto significa que, a menos que se indique lo contrario, una clase no puede ser heredada y un método no puede ser sobrescrito.

El modificador ***open*** es el que nos permite anular este comportamiento final y poder implementar la herencia.

open class	Permite que otras clases hereden de ella (que sea clase padre).
open fun	Permite que una clase hija sobrescriba (use override) este método.
open val / var	Permite que una clase hija sobrescriba esta propiedad.

La palabra **override** le indica al compilador que la función o propiedad que se está declarando en la *clase hija* intenta reemplazar (sobrescribir) una función o propiedad que ya existe en la *clase padre*.

Si no se usa override cuando se intenta cambiar el comportamiento de un miembro heredado, el compilador generará un error. Si se usa override pero el miembro del padre no está marcado como open (o abstract), también se generará un error.

Herencia

```
open class Animal(val nombre: String)
   open fun comer(alimento: String) {
      println("$nombre está comiendo $alimento.")
   }
}
```

```
class Perro(nombre: String) : Animal(nombre)
   override fun comer(alimento: String) {
      super.comer(alimento) println("$nombre lame el plato.")
   }
}
```

```
fun main() {
    val animalGenerico = Animal(“Bicho")
    val miPerro = Perro("Firulais")
    animalGenerico.comer("hierba")
    miPerro.comer(“un hueso")
}
```




```
Bicho está comiendo hierba.
Firulais está comiendo un hueso
Firulais lame el plato.
```

Herencia

Sobrescribir Propiedades

Al igual que en la sobrecritura de métodos, sobrescribir una propiedad requiere usar el modificador **override** sobre la propiedad en la subclase.

Si la propiedad está declarada con val en la clase padre, es posible reescribirla con var en la clase hija. Sin embargo, lo contrario no es posible y da error de compilación.

```
open class Animal(open val nombre: String) {
    open fun comer(alimento: String) {
      println("$nombre está comiendo $alimento.")
   }…
}
```

```
class Perro(nombrePerro: String) : Animal(nombrePerro)
   override var nombre: String = nombrePerro

    fun cambiarNombre(nuevoNombre: String) {
       nombre = nuevoNombre
       println("El gato ahora se llama $nombre.")
    }
}
```

Herencia

Herencia con constructor primario y secudario

Si **Perro** tiene un constructor primario, se hace la llamada al constructor de la clase padre en el encabezado.

```
class Perro(nombre: String, edad: Int) : Animal(nombre, edad) { … }
```

Si **Perro** tiene un constructor secundario y quiero usarlo, la inicialización del padre siempre debe ocurrir, y el secundario delega esta responsabilidad de una de estas dos maneras:

- Si además tiene un constructor primario, el constructor secundario hace una llamada a este con **this()** y va en el encabezado.

```
class Perro(nombre: String, edad: Int) : Animal(nombre, edad) {
    var estaDormido: Boolean = false
    constructor(nombre: String, edad: Int, dormido: Boolean) : this(nombre, edad) {
      this.estaDormido = dormido
 }
```

- Si tiene no tiene un constructor primario, se hace una llamada al constructor de la clase padre mediante **super()** y va en el encabezado.

```
class Perro : Animal {
     var estaDormido: Boolean = false
     constructor(nombre: String, edad: Int, dormido: Boolean) : super(nombre, edad) {
     this.estaDormido = dormido
 }
```

Herencia

Ejemplo

```
enum class Raza {
    LABRADOR,
    PASTOR_ALEMAN,
    YORKSHIRE,
    OTRA
}
```

```
open class Animal(val nombre: String, val edad: Int) {
    val esAdulto: Boolean
        get() = this.edad > 3
    var peso: Double = 0.0
    val sobrepeso: Boolean
        get() = this.peso > 50.0
    var estaDormido: Boolean = false

    fun obtenerTipoClase(): String {
        return this::class.simpleName ?: "Animal Desconocido"
    }
}
```

```
class Gato(nombre: String, edad: Int) : Animal(nombre, edad) {
     var tienePelaje: Boolean: false
}
```

```
class Perro(nombre: String, edad: Int) : Animal(nombre, edad) {
    var raza: Raza? = null
    val numPatas: Int = 4
    constructor(nombre: String, edad: Int, raza: Raza) : this(nombre, edad) {
        this.raza = raza
    }
 }
```

```
class Pez : Animal {
     var esTropical: Boolean = false
     constructor(nombre: String, edad: Int, trop: Boolean) : super(nombre, edad) {
      this.esTropical = trop
     }
 }
```

```
fun main() {
    val animal = Animal("Nombre Animal", 10)
    val michi = Gato("Michi", 3)
    val firu = Perro("Firulais", 5, Raza.YORKSHIRE)
    val nemo = Pez("Nemo", 1, true)

    val inventario = listOf(animal, michi, firu, nemo)

    println("***** Animales *****\n")
    for (a in inventario) {
        var tipo = a.obtenerTipoClase()
        val mensaje = "Hay un $tipo que se llama \"${a.nombre}\""
        println("$mensaje")
    }
}
```




```
Hay un Animal que se llama "Nombre Animal"
Hay un Gato que se llama "Michi"
Hay un Perro que se llama "Firulais"
Hay un Pez que se llama "Nemo"
```

Interfaces

Las interfaces en Kotlin son una característica fundamental de la Programación Orientada a Objetos (POO) que permite definir funcionalidad para diferentes objetos de diferentes clases.

A diferencia de las clases, una interfaz no puede ser instanciada directamente, sino que especifica qué métodos y propiedades deben implementar las clases que la implementan.

Al igual que en Java, una interfaz te dice qué debe hacer un objeto, sin preocuparte por cómo lo hace.

A diferencia de Java, en Kotlin no es necesario utilizar la palabra abstract. Cualquier método o atributo declarado en una interfaz sin un cuerpo o sin un valor de inicialización es **implícitamente** abstract. Esto significa que cualquier clase que implemente la interfaz **debe tener** una implementación para estos métodos/atributos. Cuando una clase los implementa, **debe** usar la palabra clave **override**. Si no los implementa, hará uso de los definidos en la interfaz.

Otra diferencia con Java es que las interfaces de Kotlin pueden contener implementaciones de métodos con cuerpo. Las clases pueden usar esta implementación por defecto o anularla.

Interfaces

```
open class Animal(val nombre: String, val edad: Int) {
    val esAdulto: Boolean
        get() = this.edad > 3
    var peso: Double = 0.0
    val sobrepeso: Boolean
        get() = this.peso > 50.0
    var estaDormido: Boolean = false
        private set
    fun obtenerTipoClase(): String {
        return this::class.simpleName ?: "Animal Desconocido"
    }

    open fun comer(alimento: String) {
        if (!estaDormido) {
            println("\n${nombre} está comiendo $alimento.")
        } else {
            println("\n${nombre} no puede comer, está durmiendo.")
        }
    }

    fun dormir() {
        if (!estaDormido) {
            println("\n${nombre} se fue a dormir.")
            estaDormido = true
        }
    }

    fun despertar() {
        if (estaDormido) {
            println("\n${nombre} se ha despertado.")
            estaDormido = false
        } else {
            println("\n${nombre} ya esta despierto.")
        }
    }
}
```

```
interface FuncionesAnimal {
    fun emitirSonido() // Método abstracto: cada animal debe definir su sonido.
    fun desplazarse() {// Método con implementación por defecto.
        println("El animal se desplaza.")
    }
}
```

```
class Perro(nombre: String, edad: Int) : Animal(nombre, edad), FuncionesAnimal {
    var raza: Raza? = null
    val numPatas: Int = 4
    constructor(nombre: String, edad: Int, raza: Raza) : this(nombre, edad) {
        this.raza = raza
    }

    // Implementación obligatoria del metodo vacío en la interfaz emitirSonido
    override fun emitirSonido() {
        println("¡${nombre} ladra: Guau-Guau!")
    }

    // Sobrescritura de desplazarse() que tiene codigo en la interfaz
    override fun desplazarse() {
        if (!estaDormido) {
            println("${nombre} está corriendo y saltando.")
        } else {
            println("${nombre} no puede desplazarse, está durmiendo.")
        }
    }

    // Metodo comer para el perro, sobreescribe el de la clase padre Animal
    override fun comer(alimento: String) {
        if (alimento == "chocolate") {
            println("¡Cuidado! ${nombre} no debe comer $alimento.")
        } else {
            super.comer(alimento)
        }
    }
}
```

Interfaces

```
fun main() {
    val firu = Perro("Firulais", 5)

    var tipo = firu::class.simpleName
    println("${firu.nombre} es un ${tipo!!.lowercase()} que tiene ${firu.numPatas} patas.")

    println("\n***** ¿ Qué hace ${firu.nombre} ? *****")

    firu.desplazarse()

    firu.comer("croquetas")
    firu.emitirSonido()

    firu.dormir()
    firu.comer("pienso")
    firu.comer("chocolate")

    firu.despertar()
    firu.comer("una lata")
    firu.comer("chocolate")
}
```




```
Firulais es un perro que tiene 4 patas.

***** ¿ Qué hace Firulais ? *****
Firulais está corriendo y saltando.

Firulais está comiendo croquetas.
¡Firulais ladra: Guau-Guau!

Firulais se fue a dormir.

Firulais no puede comer, está durmiendo.
¡Cuidado! Firulais no debe comer chocolate.

Firulais se ha despertado.

Firulais está comiendo una lata.
¡Cuidado! Firulais no debe comer chocolate.
```

EJERCICIOS

1. Crear una clase que almacene los lados de un triángulo e implemente los siguientes métodos:

- método para inicializar las propiedades
- método para imprimir el valor del lado mayor
- método que muestre si es equilátero o no

El constructor principal debe recibir los valores de los lados y un segundo constructor que permita introducir por teclado los tres lados con las respectivas validaciones.

2. Crear una clase llamada Dado. Definir una propiedad llamada valor que permita cargar un valor comprendido entre 1 y 6 si se intenta asignar un valor que no está comprendido en este rango se debe cargar un 1.

El constructor debe recibir el valor inicial que debe tener el dado, validando la entrada.

Adicionalmente, tendrá dos métodos:

- método tirar(). Simula el lanzamiento del dado. Debe generar un número entero aleatorio comprendido entre 1 y 6 (ambos inclusive) y asignar este nuevo valor a la propiedad valor.
- método mostrar(). Muestra por consola el valor actual de la propiedad valor.